University Examination Answers

Django Web Framework

Q1. Define Web Framework. Explain with an example of CGI and its disadvantages.

Definition of Web Framework

A Web framework provides a programming infrastructure for your applications, so that you can focus on writing clean, maintainable code without having to reinvent the wheel. Django is a prominent member of a new generation of Web frameworks.

CGI Example (circa 1998)

To understand what a web framework solves, consider a Web application written using the Common Gateway Interface (CGI) standard. Below is a simple CGI script in Python that displays the ten most recently published books from a database:

```
#!/usr/bin/python
import MySQLdb
print "Content-Type: text/html"
print
print "<html><head><title>Books</title></head>"
print "<body>"
print "<h1>Books</h1>"
print "<ul>"
connection = MySQLdb.connect(user='me', passwd='letmein', db='my_db')
cursor = connection.cursor()
cursor.execute("SELECT name FROM books ORDER BY pub_date DESC LIMIT 10")
for row in cursor.fetchall():
    print "<li>%s</li>" % row[0]
print "</ul>"
print "</body></html>"
connection.close()
```

Disadvantages of CGI

- Database code duplication: When multiple pages need to connect to the database, that database-connecting code would be duplicated in each individual CGI script.
- Boilerplate overhead: Developers must worry about printing the 'Content-Type' line and remembering to close the database connection, reducing productivity and introducing opportunities for mistakes.
- No environment-specific configuration: When code is reused in multiple environments with separate databases and passwords, there is no clean configuration approach.
- No separation of logic and design: A Web designer cannot redesign the page without touching the Python logic, because the logic of retrieving from the database is mixed with the HTML display code.

These problems are precisely what a Web framework intends to solve. Django solves all of these by providing reusable components, URL routing, template system, and ORM.

Q2. Explain how Django Processes a Request with a flow diagram.

When a request comes in, Django follows a well-defined processing pipeline:

Step-by-Step Explanation

- The command python manage.py runserver imports settings.py. The ROOT_URLCONF setting tells Django which Python module to use as the URLconf.
- When a request arrives (e.g., /time/), Django loads the URLconf pointed to by ROOT_URLCONF.
- Django checks each URLpattern in the URLconf in order, comparing the requested URL with patterns one at a time until it finds a match.
- When a match is found, Django calls the associated view function, passing an HttpRequest object as the first parameter.
- Request and View middleware can augment or short-circuit the processing. If middleware returns an HttpResponse, the view is bypassed.
- If the view raises an exception, control passes to Exception middleware.
- The view function is responsible for returning an HttpResponse object.
- Response middleware performs post-processing before the response is sent to the browser.

Flow Diagram:

Browser sends HTTP Request

↓

Server-specific Handler constructs HttpRequest

↓

Request Middleware (augments/short-circuits?)

↓

URLconf Resolution (ROOT_URLCONF → match URLpattern)

↓

View Middleware

↓

View Function called (returns HttpResponse or raises exception)

↓

Exception Middleware (if exception raised)

↓

Response Middleware (post-processing)

↓

HTTP Response sent to Browser

Q3. Explain Wildcard URL patterns and Django's Pretty Error Pages.

Wildcard URL Patterns

Wildcard URL patterns allow a single URLpattern to match multiple URLs by using regular expressions. This avoids writing separate view functions and URLconf entries for every possible input.

Example: To display current time offset by any number of hours, instead of defining:

```
urlpatterns = patterns('',
    (r'^time/plus/1/$', one_hour_ahead),
    (r'^time/plus/2/$', two_hours_ahead),
    ...  # This approach is flawed!
)
```

Django uses a wildcard with \d+ (one or more digits):

```
urlpatterns = patterns('',
    (r'^time/$', current_datetime),
    (r'^time/plus/(\d{1,2})/$', hours_ahead),
)
```

The pattern \d{1,2} limits the offset to 1 or 2 digits (max 99 hours). Parentheses around \d{1,2} capture the matched value and pass it to the view function as the offset parameter.

The corresponding view function:

```
def hours_ahead(request, offset):
    offset = int(offset)
    dt = datetime.datetime.now() + datetime.timedelta(hours=offset)
    html = "<html><body>In %s hour(s), it will be %s.</body></html>" % (offset, dt)
    return HttpResponse(html)
```

- The offset parameter receives the captured string from the URL pattern.
- int(offset) converts the string to integer for use in timedelta.
- URLconfs provide input validation — since the pattern only captures digits, int() is safe.

Django's Pretty Error Pages

Django provides informative error pages during development (when DEBUG=True). These include:

- Exception type and message at the top of the page (e.g., TypeError: unsupported type for timedelta hours component: str).
- Full Python traceback with file names, function/method names, line numbers, and source code for each stack frame.
- Interactive 'Local vars' section under each stack frame showing all local variables and their values at the point the exception was raised.
- 'Switch to copy-and-paste view' for sharing tracebacks for technical support.
- 'Request information' section showing GET, POST, cookie values, and CGI headers.
- 'Settings' section listing all Django installation settings.

Important: The error page is only displayed when DEBUG=True. In production (DEBUG=False), Django shows a generic 500 error page, protecting sensitive information from public exposure.

For 404 errors, Django's debug page also shows exactly which URLconf was used and every pattern in it, helping developers identify why the URL was not matched.

Q4. Write a Django app that displays date/time four hours ahead and four hours before as an offset of current date/time.

views.py

```
from django.http import HttpResponse
import datetime

def time_offsets(request):
    now = datetime.datetime.now()
    four_ahead = now + datetime.timedelta(hours=4)
    four_before = now - datetime.timedelta(hours=4)
    html = (
        "<html><body>"
        "<h2>Time Offsets</h2>"
        "<p><b>Current Time:</b> %s</p>"
        "<p><b>4 Hours Ahead:</b> %s</p>"
        "<p><b>4 Hours Before:</b> %s</p>"
        "</body></html>"
    ) % (now, four_ahead, four_before)
    return HttpResponse(html)
```

urls.py

```
from django.conf.urls.defaults import *
from mysite.views import time_offsets

urlpatterns = patterns('',
    (r'^time/offsets/$', time_offsets),
)
```

Explanation

- datetime.datetime.now() gets the current server date and time.
- datetime.timedelta(hours=4) creates a time difference of 4 hours.
- Adding timedelta gives time 4 hours ahead; subtracting gives time 4 hours before.
- An HttpResponse is returned containing the formatted HTML output.
- Visit http://127.0.0.1:8000/time/offsets/ to see the result.

Q5. Explain the MVC (Model-View-Controller) design pattern. How does Django implement this pattern?

MVC Design Pattern

The Model-View-Controller (MVC) pattern defines a way of developing software so that the code for defining and accessing data (the Model) is separate from request routing logic (the Controller), which in turn is separate from the user interface (the View).

- Model: Refers to the data access layer — how to access it, validate it, its behaviors, and relationships.
- View: Refers to the part of the system that selects what to display and how to display it.
- Controller: Refers to the part of the system that decides which view to use depending on user input.

The goal is loose coupling — each component can be changed independently without affecting the others.

Django's Implementation (MTV)

Django follows MVC closely enough to be called an MVC framework, but calls its pattern MTV (Model-Template-View):

- M (Model): Handled by Django's database layer (models.py). Contains data access logic, validation, and relationships.
- T (Template): The presentation layer. Contains presentation-related decisions — how something is displayed.
- V (View): The business logic layer (views.py). Contains logic that accesses the model and defers to the appropriate template.
- C (Controller): Handled by the framework itself — Django's URLconf dispatches requests to the right view.

Django Example:

```
# models.py — the Model
from django.db import models
class Book(models.Model):
    name = models.CharField(maxlength=50)
    pub_date = models.DateField()

# views.py — the View
from django.shortcuts import render_to_response
from models import Book
def latest_books(request):
    book_list = Book.objects.order_by('-pub_date')[:10]
    return render_to_response('latest_books.html', {'book_list': book_list})

# urls.py — the Controller
urlpatterns = patterns('',
    (r'latest/$', views.latest_books),
)

# latest_books.html — the Template
{% for book in book_list %}
    <li>{{ book.name }}</li>
{% endfor %}
```

A key advantage: components are loosely coupled. A designer can change the HTML template without touching Python code; a developer can change URL patterns without changing view logic; a DBA can rename a database table in one place.

Q6. Discuss the process of mapping URLs to views in Django with an example.

URL Mapping via URLconfs

A URLconf is like a table of contents for your Django-powered Web site. It is a mapping between URL patterns and the view functions that should be called for those URL patterns. It tells Django: 'For this URL, call this code, and for that URL, call that code.'

Process

- Step 1: Django's settings.py contains ROOT_URLCONF, pointing to the URLconf module (e.g., mysite.urls).
- Step 2: When a request arrives, Django loads the URLconf and checks each URLpattern in order.
- Step 3: The requested URL is compared against each regular expression pattern, one at a time.

- Step 4: When a match is found, Django calls the corresponding view function, passing an HttpRequest object.
- Step 5: The view returns an HttpResponse, which Django sends back to the browser.

Example

Create views.py:

```
from django.http import HttpResponse
import datetime

def current_datetime(request):
    now = datetime.datetime.now()
    html = "<html><body>It is now %s.</body></html>" % now
    return HttpResponse(html)
```

Create urls.py:

```
from django.conf.urls.defaults import *
from mysite.views import current_datetime

urlpatterns = patterns('',
    (r'^time/$', current_datetime),
)
```

The pattern r'^time/$' matches the URL /time/ and calls the current_datetime view. The ^ means 'start of string' and $ means 'end of string', ensuring exact matching.

Q7. Describe loose coupling in the context of Django URLConfs. Why is it important?

Loose Coupling Defined

Loose coupling is a software-development approach that values the importance of making pieces interchangeable. If two pieces of code are loosely coupled, then changes made to one of the pieces will have little or no effect on the other.

Loose Coupling in Django URLConfs

In a Django Web application, the URL definitions and the view functions they call are loosely coupled. The decision of what the URL should be for a given function, and the implementation of the function itself, reside in two separate places.

In contrast, other platforms tightly couple URLs to code:

- In typical PHP applications, the URL is designated by where you place the code on your filesystem.
- In early CherryPy, the URL corresponded to the method name in which the code lived.

This may seem convenient short-term, but becomes unmanageable long-term.

Why It Is Important

- URL changes without code changes: If you want to move a page from /time/ to /currenttime/, just edit the URLconf — no need to touch the view function.
- Logic changes without URL changes: If you alter the view function's logic, the URL it is bound to remains unaffected.
- Multiple URLs, same view: You can expose a single view at multiple URLs just by adding entries to the URLconf, without touching the view code.

- Team collaboration: URL designers and view developers can work independently.

This loose coupling makes Django applications clean, maintainable, and flexible over time.

Q8. What are wildcard patterns in URLs? Provide an example of how they are used in Django.

Wildcard URL Patterns

A wildcard URL pattern uses regular expression syntax to match variable parts of a URL, rather than requiring an exact fixed string. This allows a single URLpattern and a single view function to handle a range of URLs.

Example

Consider a URL that accepts any 1 or 2-digit number representing hours:

```
from django.conf.urls.defaults import *
from mysite.views import current_datetime, hours_ahead

urlpatterns = patterns('',
    (r'^time/$', current_datetime),
    (r'^time/plus/(\d{1,2})/$', hours_ahead),
)
```

Here \d{1,2} is a regular expression that matches one or two digits. The parentheses capture the matched value and pass it to the view as a parameter.

The view function receives the captured value:

```
def hours_ahead(request, offset):
    offset = int(offset)   # Convert captured string to integer
    dt = datetime.datetime.now() + datetime.timedelta(hours=offset)
    html = "<html><body>In %s hour(s), it will be %s.</body></html>" % (offset, dt)
    return HttpResponse(html)
```

- /time/plus/3/ → offset = '3'
- /time/plus/21/ → offset = '21'
- /time/plus/100/ → No match (3 digits exceed \d{1,2}), returns 404

This also provides input validation: since the pattern only captures digits, int() conversion will never fail.

Q9. Explain the evolution of Django and mention any two key features.

Evolution of Django

Django grew organically from real-world applications. Its evolution can be traced through these milestones:

- Fall 2003: Born at the Lawrence Journal-World newspaper in Lawrence, Kansas. Web programmers Adrian Holovaty and Simon Willison began using Python to build applications under extreme journalism deadlines.
- 2003–2005: The World Online team (responsible for LJWorld.com, Lawrence.com, KUsports.com) developed a time-saving framework out of necessity — features had to be added in days or even hours.

- Summer 2005: Jacob Kaplan-Moss joined, and the team decided to release the framework as open source software. Released in July 2005, named 'Django' after jazz guitarist Django Reinhardt.
- Post-2005: Django grew into a global open source project with contributors worldwide. The original World Online developers provide central guidance, while World Online contributes employee time, marketing materials, and hosting for the Django website.

The classic path Django followed mirrors what most developers experience: writing apps, noticing repeated patterns, refactoring shared code, and eventually inventing a framework.

Two Key Features of Django

- Admin Interface: Django offers a built-in, automatically generated admin interface — particularly well suited for 'content' sites like news portals, e-commerce platforms, and database-driven sites. This was born from the newsroom context where rapid content management was essential.
- Loose Coupling and Reusability: Django is designed for loose coupling and strict separation of concerns — URL configuration, view logic, templates, and models are all independent, making applications easy to maintain and evolve. The framework's developers are acutely focused on solving real-world Web development problems, resulting in active improvements on an almost daily basis.

Q10. Discuss the concept of views in Django.

What is a View?

A view function, or view for short, is simply a Python function that takes a Web request and returns a Web response. This response can be:

- The HTML contents of a Web page
- A redirect
- A 404 error
- An XML document
- An image or any other content

The view itself contains whatever arbitrary logic is necessary to return that response. It can live anywhere on the Python path — there is no magic requirement about location.

Structure of a View

```
from django.http import HttpResponse
import datetime

def current_datetime(request):
    now = datetime.datetime.now()
    html = "<html><body>It is now %s.</body></html>" % now
    return HttpResponse(html)
```

- Every view takes an HttpRequest object as its first parameter (typically named request).
- The view function name does not matter to Django — it is referenced by the URLconf.
- Every view must return an HttpResponse object.
- The view contains the business logic layer in the MTV pattern.

Role of Views in Django's MTV Pattern

In Django's MTV (Model-Template-View) pattern:

- Views serve as the bridge between Models (data) and Templates (presentation).

- Views access the model to retrieve data, apply business logic, and then pass data to the appropriate template.
- Views are loosely coupled from URLs — the URL that triggers a view is defined separately in the URLconf, not inside the view function itself.

Views are connected to URLs via URLconfs using the ROOT_URLCONF setting, making them interchangeable and independently testable.

Q11. Write a Python function in Django framework that displays date and time which is an offset of n hours from current date and time.

views.py

```
from django.http import HttpResponse
import datetime

def hours_ahead(request, offset):
    """
    Displays date and time that is 'offset' hours from the current server time.
    The offset value is captured from the URL pattern.
    """
    offset = int(offset)   # Convert captured string to integer
    dt = datetime.datetime.now() + datetime.timedelta(hours=offset)
    html = (
        "<html><body>"
        "<h2>Time Offset Calculator</h2>"
        "<p>Offset: %s hour(s)</p>"
        "<p>Resulting Date and Time: %s</p>"
        "</body></html>"
    ) % (offset, dt)
    return HttpResponse(html)
```

urls.py

```
from django.conf.urls.defaults import *
from mysite.views import hours_ahead

urlpatterns = patterns('',
    (r'^time/plus/(\d{1,2})/$', hours_ahead),
)
```

Explanation

- request: An HttpRequest object — always the first parameter of any Django view.
- offset: The string captured by the parentheses in the URLpattern (\d{1,2}).
- int(offset): Converts the captured string (e.g., '3') into an integer, required by timedelta.
- datetime.timedelta(hours=offset): Represents a duration of offset hours.
- datetime.datetime.now() + timedelta: Computes the future/past time depending on sign.
- HttpResponse(html): Returns the HTML page to the browser.

Sample URLs and Output

- /time/plus/3/ → Shows time 3 hours from now
- /time/plus/24/ → Shows time 24 hours from now

- /time/plus/100/ → Returns 404 (exceeds \d{1,2} pattern)

Note: To show time in the past, change the operator to minus (–). The URLconf pattern only allows positive offsets; negative offsets would require a different URL pattern design.